Infrastructure Monitoring and Alerting Dashboard Using Docker, Prometheus, and Grafana

1. Project Overview

An Infrastructure Monitoring and Alerting Dashboard was implemented on an AWS EC2 instance using Docker Compose, Prometheus, Grafana, and Node Exporter.

The project monitors CPU utilization, memory utilization, disk utilization, and network traffic through a centralized Grafana dashboard. Prometheus collects and stores infrastructure metrics, while Grafana visualizes the metrics and evaluates alert rules. Email notifications are sent when configured resource utilization thresholds are exceeded.

The implementation also includes persistent storage, secure access through an SSH tunnel, email notification testing, and service restart verification.

2. Objectives

- Deploy a monitoring environment using Docker Compose.
- Collect infrastructure metrics using Prometheus and Node Exporter.
- Visualize system resource utilization through Grafana.
- Configure alerts for high CPU, memory, and disk utilization.
- Configure email notifications using Gmail SMTP.
- Test alert triggering and email delivery.
- Verify service recovery and dashboard availability after container restarts.
- Apply basic security measures and manage AWS resources carefully.

3. Technologies Used

Technology	Purpose
AWS EC2	Hosted the monitoring environment
Red Hat Enterprise Linux 10.2	Operating system
Docker CE 29.9.0	Container runtime

Docker Compose v5.6.0	Container orchestration
Prometheus	Metrics collection and storage
Grafana 12.3.0	Dashboards and alert management
Node Exporter	Host-level metrics exporter
Gmail SMTP	Email alert delivery
SSH	Secure remote access

4. AWS Infrastructure Setup

An AWS EC2 instance was configured to host the monitoring environment.

Instance details

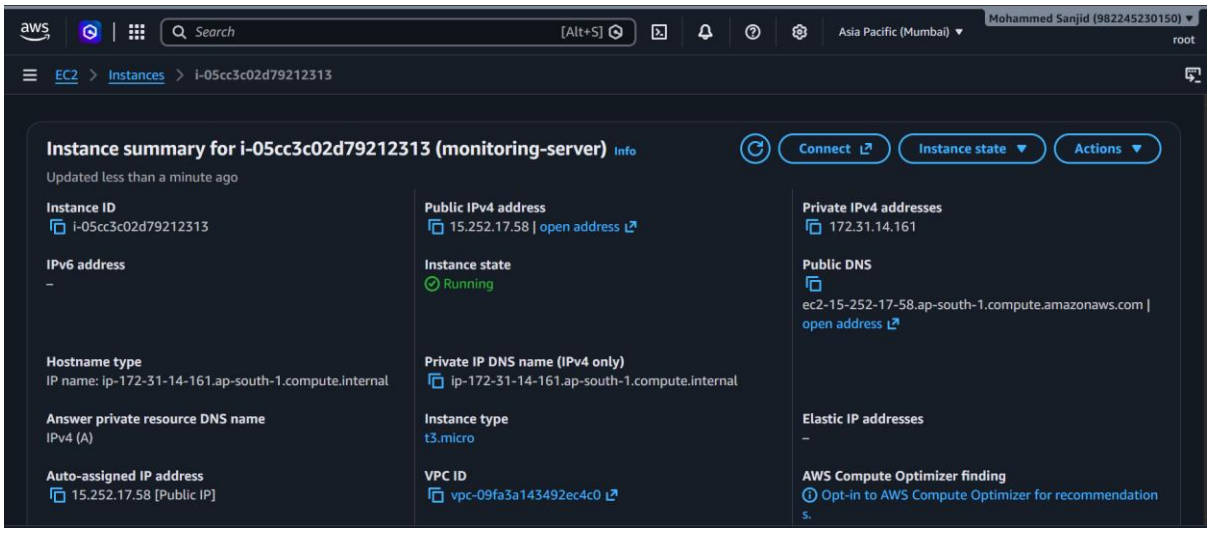
- Instance type: `t3.micro`
- Operating system: Red Hat Enterprise Linux 10.2
- vCPUs: 2
- Memory: Approximately 1 GiB
- Root storage: 10 GiB EBS volume
- AWS Region: Asia Pacific (Mumbai), `ap-south-1`

Docker and Docker Compose were installed, and the project files were maintained in:

`/home/ec2-user/monitoring-dashboard`

The monitoring web interfaces were bound to the instance's loopback interface, preventing direct public access to Grafana and Prometheus.

Screenshot 1: AWS EC2 instance details



5. Project Directory Structure

The project was organized into the following directory structure:

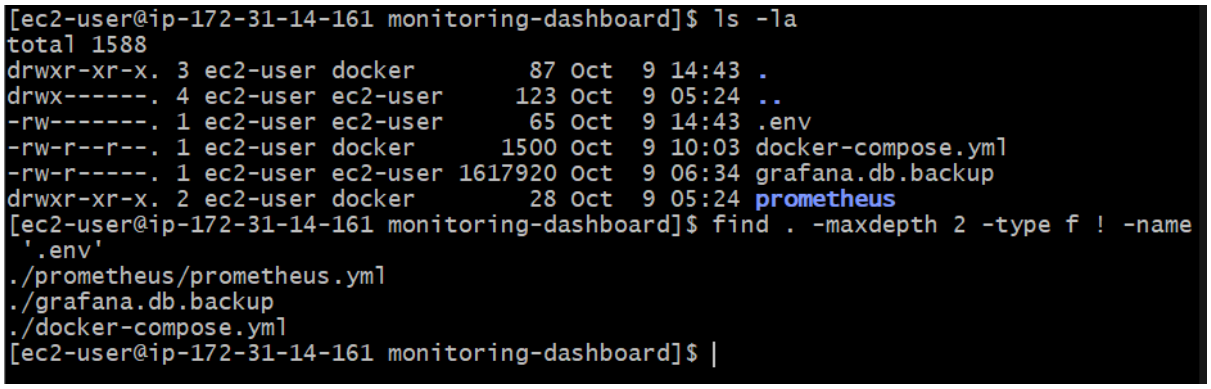
```
monitoring-dashboard/
├── docker-compose.yml
├── .env
├── grafana.db.backup
└── prometheus/
    └── prometheus.yml
```

File descriptions

- `docker-compose.yml`: Defines the Grafana, Prometheus, and Node Exporter services.
- `.env`: Stores sensitive environment variables for Grafana administration and SMTP authentication.
- `prometheus/prometheus.yml`: Defines the Prometheus scrape configuration.
- `grafana.db.backup`: Contains a backup copy of the Grafana database.

The `.env` file was protected with file permissions `600`, restricting access to the file owner. Sensitive passwords were not included in the documentation.

Screenshot 2: Project directory structure



6. Docker Compose Deployment

Docker Compose was used to manage the three monitoring services:

1. Prometheus
2. Grafana
3. Node Exporter

The Compose configuration included persistent named volumes for Prometheus and Grafana data. Memory limits and Prometheus data retention settings were configured to help manage resource usage on the EC2 instance.

The Grafana image was pinned to version `12.3.0`. The Prometheus and Node Exporter services used their official container images.

The configuration was validated using:

```
cd ~/monitoring-dashboard
docker compose config -q
```

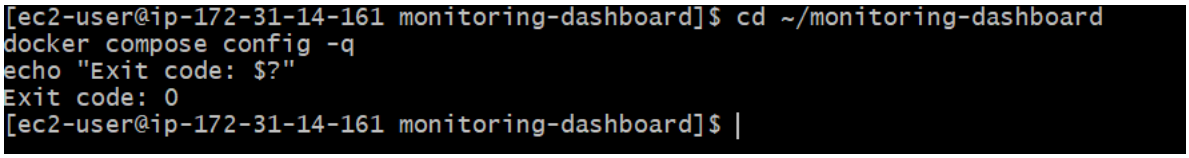
The command completed without errors.

The status of the containers was verified using:

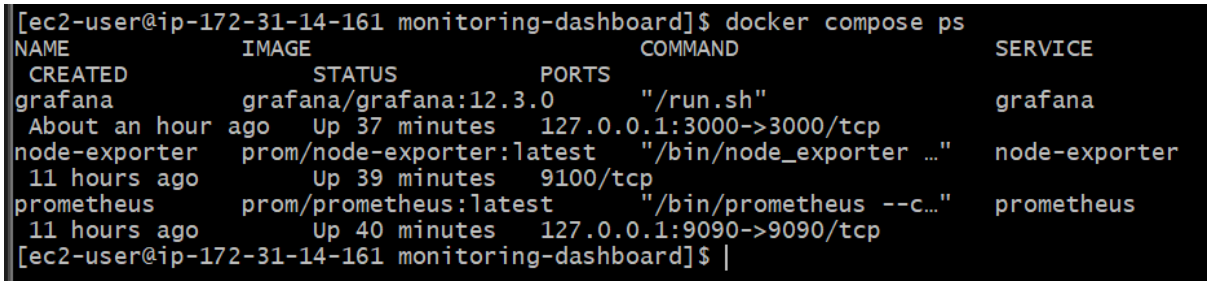
```
docker compose ps
```

All three containers were confirmed to be running.

Screenshot 3: Docker Compose configuration validation



Screenshot 4: Running containers



7. Prometheus Configuration

Prometheus was configured to scrape metrics every 30 seconds, with a 30-second evaluation interval. Data retention was limited to one day or approximately 300 MB, whichever limit was reached first.

The following targets were configured:

Job	Target	Purpose
Prometheus	`prometheus:9090`	Monitored Prometheus itself
Node Exporter	`node-exporter:9100`	Collected host system metrics

The configuration file was stored at:

`~/monitoring-dashboard/prometheus/prometheus.yml`

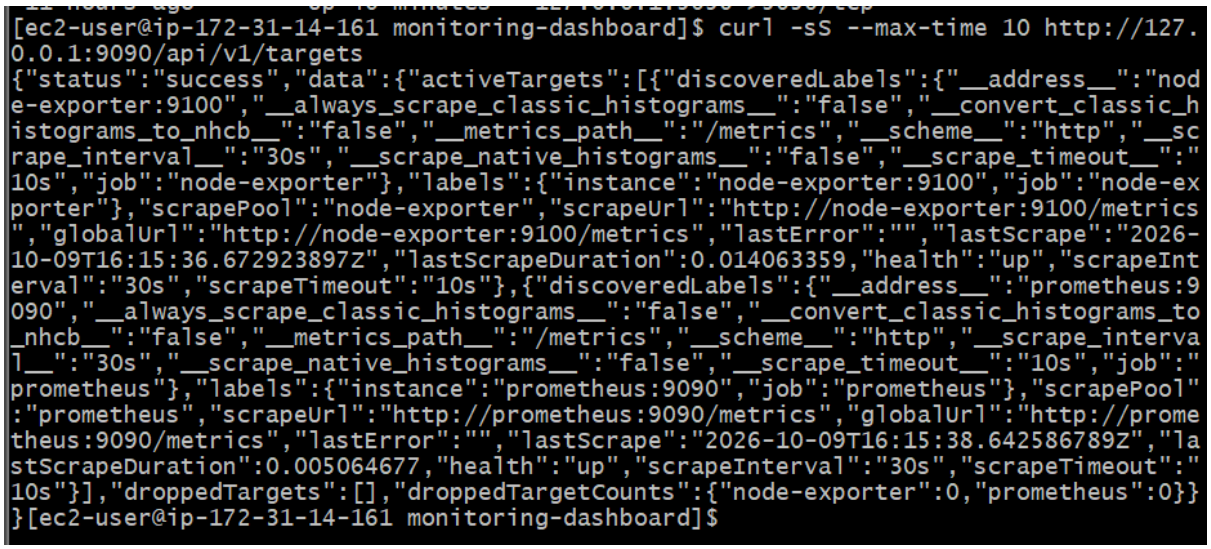
Prometheus used the `prometheus-data` named volume for persistent storage.

The target status was checked using:

```
curl -sS --max-time 10 \
  http://127.0.0.1:9090/api/v1/targets
```

The API response confirmed that both targets reported `health: up` without scrape errors.

Screenshot 5: Prometheus target health



8. Node Exporter Configuration

Node Exporter was deployed as a Docker container to expose host-level metrics, including CPU activity, memory utilization, filesystem usage, and network traffic.

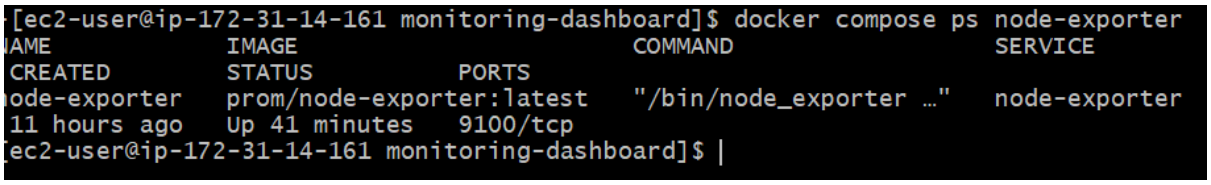
The container was configured to access host system information through a read-only root filesystem mount and the host PID namespace.

Prometheus collected metrics from the internal Docker network endpoint:

http://node-exporter:9100/metrics

Node Exporter's port was not published directly to the public network.

Screenshot 6: Node Exporter container



9. Grafana Configuration

Grafana version `12.3.0` was deployed to visualize the collected metrics and manage alert rules.

The Prometheus data source was configured with the URL:

http://prometheus:9090

This internal Docker service address enabled Grafana to communicate with Prometheus over the Docker network.

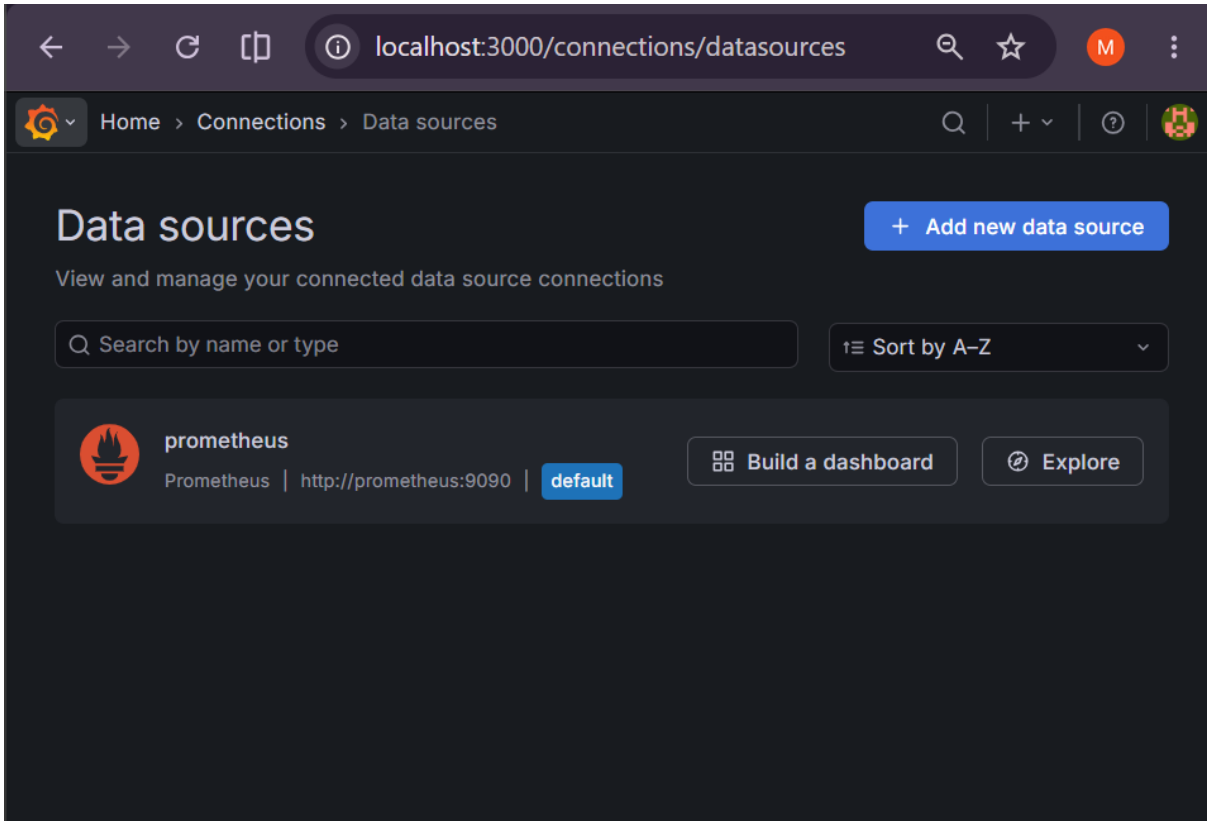
The connection was tested using Grafana's **Save & test** option and completed successfully.

Grafana's health was verified using:

```
curl -sS --max-time 10 \
  http://127.0.0.1:3000/api/health
```

The response confirmed that the Grafana database was healthy and the running version was `12.3.0`.

Screenshot 7: Grafana Prometheus data source



Screenshot 8: Grafana health check

```
[ec2-user@ip-172-31-14-161 monitoring-dashboard]$ curl -sS --max-time 10 http://127.
0.0.1:3000/api/health
{
  "database": "ok",
  "version": "12.3.0",
  "commit": "20051fb1fc604fc54aae76356da1c14612af41d0"
}[ec2-user@ip-172-31-14-161 monitoring-dashboard]$
```

10. Secure Access Through SSH Tunneling

Grafana and Prometheus were bound to the EC2 instance's loopback interface:

- Grafana: `127.0.0.1:3000`
- Prometheus: `127.0.0.1:9090`

The services were accessed from a Windows computer through an SSH tunnel.

```
ssh -i "$HOME\Downloads\sanjidnew.pem" `
  -L 3000:127.0.0.1:3000 `
  -L 9090:127.0.0.1:9090 `
  ec2-user@<EC2-PUBLIC-IP>
```

The private key path and EC2 public IP were replaced with the actual values when connecting.

After the tunnel was established, Grafana was accessed through:

http://localhost:3000

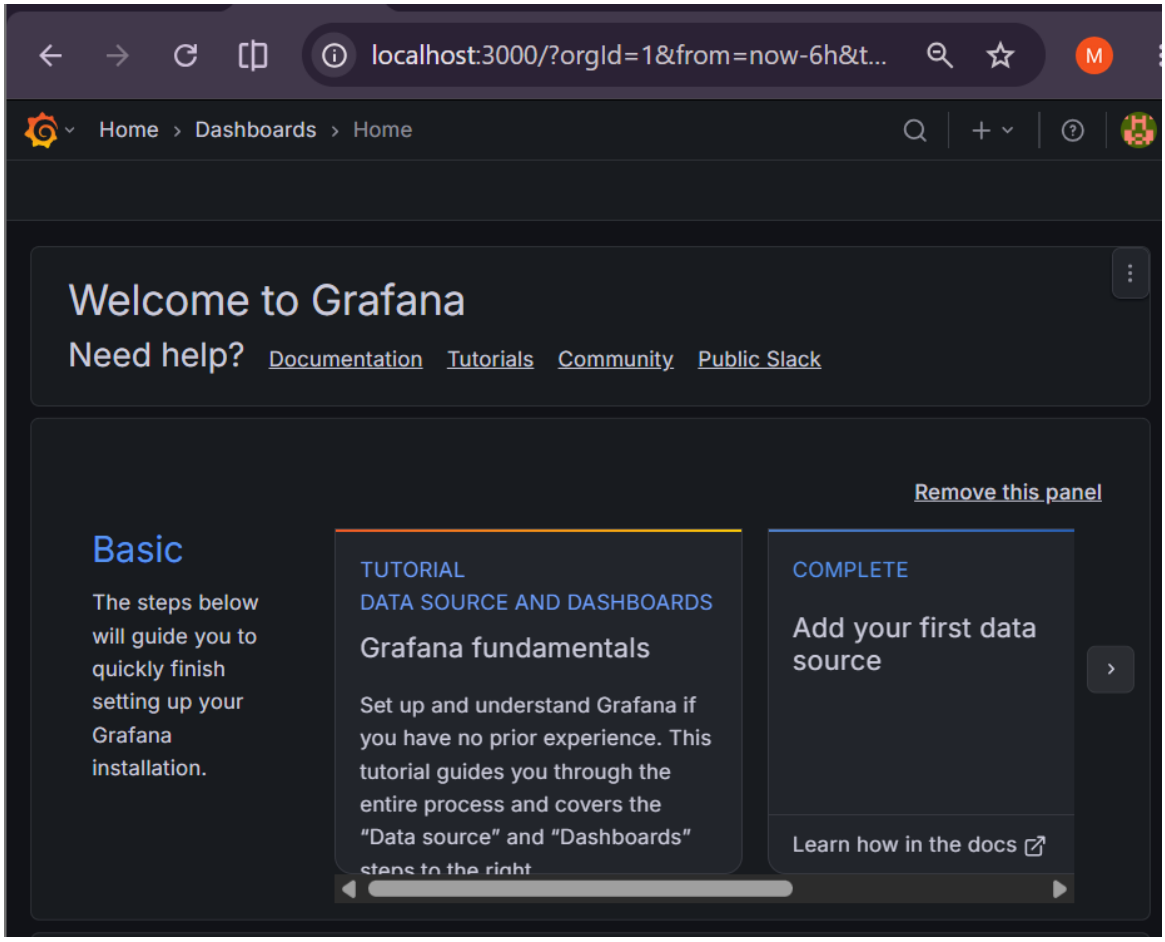
Prometheus was accessed through:

http://localhost:9090

The SSH session was kept open while using the interfaces. This approach avoided exposing the monitoring web interfaces directly to the internet.

Screenshot 9: SSH tunnel and local Grafana access





11. Infrastructure Monitoring Dashboard

A Grafana dashboard named **Infrastructure Monitoring Dashboard** was created and saved. It contained five panels for monitoring key server resources.

11.1 CPU Usage (%)

The gauge displayed CPU utilization calculated from the proportion of CPU time spent in non-idle states.

```
100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))
```

11.2 Memory Usage (%)

The gauge displayed memory utilization based on total and available system memory.

```
100 * (1 - node_memory_MemAvailable_bytes /
node_memory_MemTotal_bytes)
```

11.3 Disk Usage (%)

The gauge displayed the percentage of used space on the root filesystem.

```
100 * (
  1 -

node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs",mountpo
int="/"}
  /

node_filesystem_size_bytes{fstype!~"tmpfs|overlay|squashfs",mountpoi
nt="/"}
)
```

11.4 Network Receive (Bytes/sec)

The time-series panel displayed incoming network traffic, excluding the loopback interface.

```
rate(node_network_receive_bytes_total{device!="lo"}[5m])
```

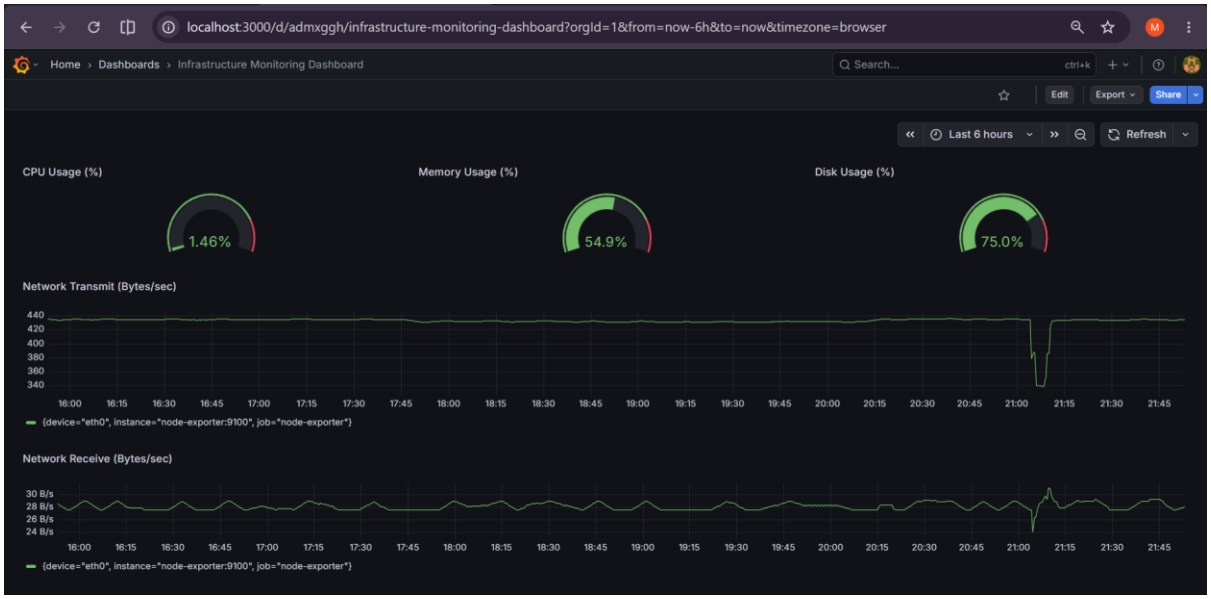
11.5 Network Transmit (Bytes/sec)

The time-series panel displayed outgoing network traffic, excluding the loopback interface.

```
rate(node_network_transmit_bytes_total{device!="lo"}[5m])
```

The three gauge panels were arranged in the upper row, while the two network traffic graphs were placed below them. The dashboard was saved and verified after restarting the monitoring services.

Screenshot 10: Infrastructure Monitoring Dashboard



12. Alert Rules Configuration

Three Grafana alert rules were configured in the `Infrastructure Monitoring` folder and the `Infrastructure Alerts` evaluation group.

The evaluation interval was one minute, and each rule used a two-minute pending period.

Alert rule	Metric	Threshold	Pending period
High CPU Usage	CPU utilization	Above 80%	2 minutes
High Memory Usage	Memory utilization	Above 85%	2 minutes
High Disk Usage	Root filesystem utilization	Above 80%	2 minutes

12.1 High CPU Usage

The rule used the CPU utilization query and triggered when CPU usage remained above 80% for two minutes.

Summary: High CPU usage detected on the EC2 monitoring server.

The description instructed the administrator to inspect the dashboard and investigate processes consuming excessive CPU resources.

12.2 High Memory Usage

The rule used the memory utilization query and triggered when memory usage remained above 85% for two minutes.

Summary: High memory usage detected on the EC2 monitoring server.

The description recommended reviewing memory availability, swap usage, and processes consuming excessive memory.

12.3 High Disk Usage

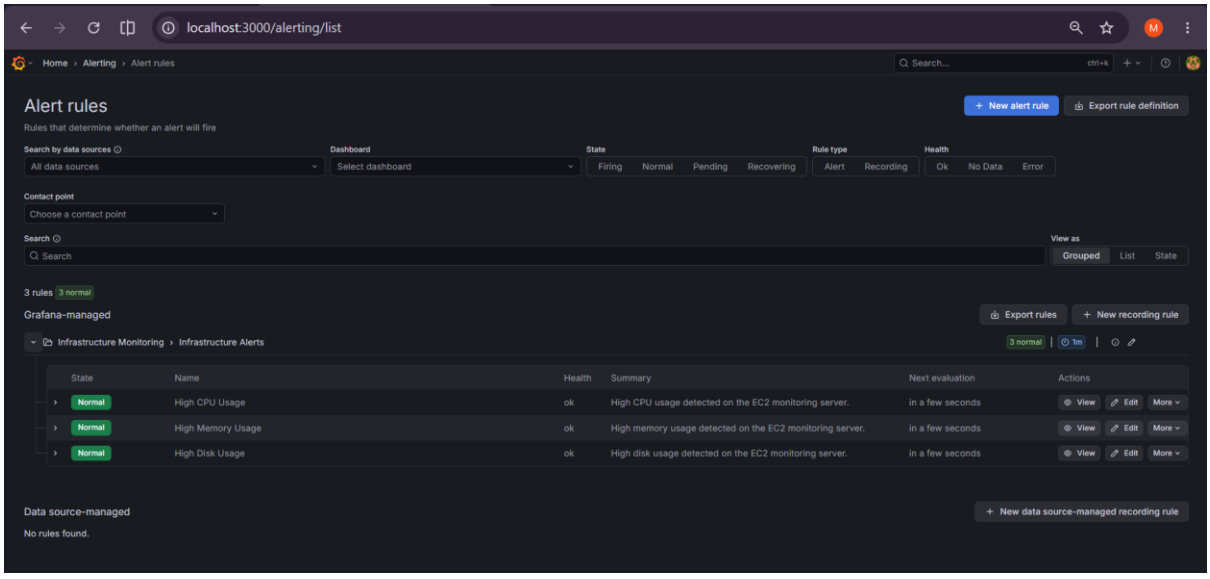
The rule used the root filesystem utilization query and triggered when disk usage remained above 80% for two minutes.

Summary: High disk usage detected on the EC2 monitoring server.

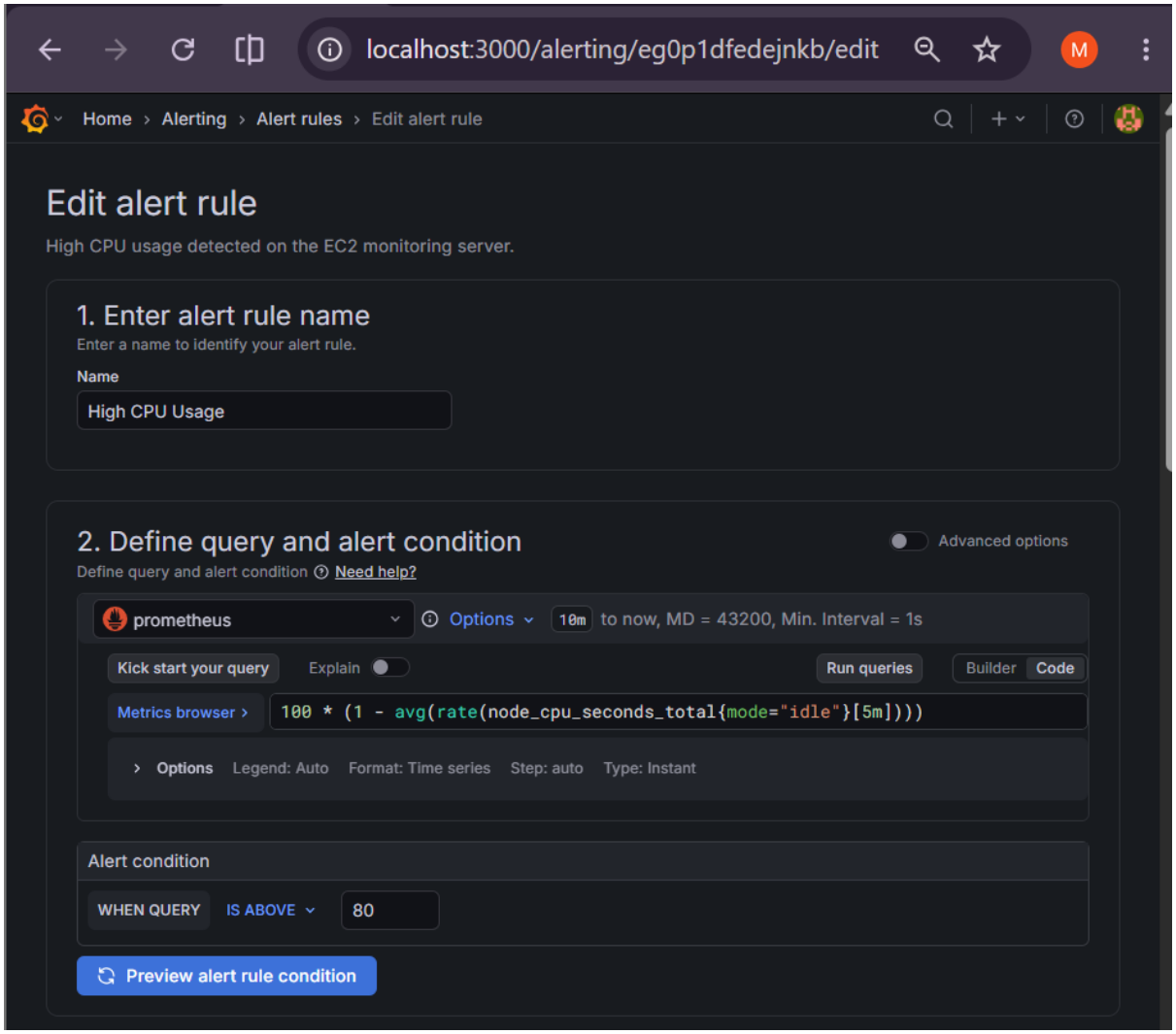
The description recommended investigating large files, logs, and unnecessary data consuming disk space.

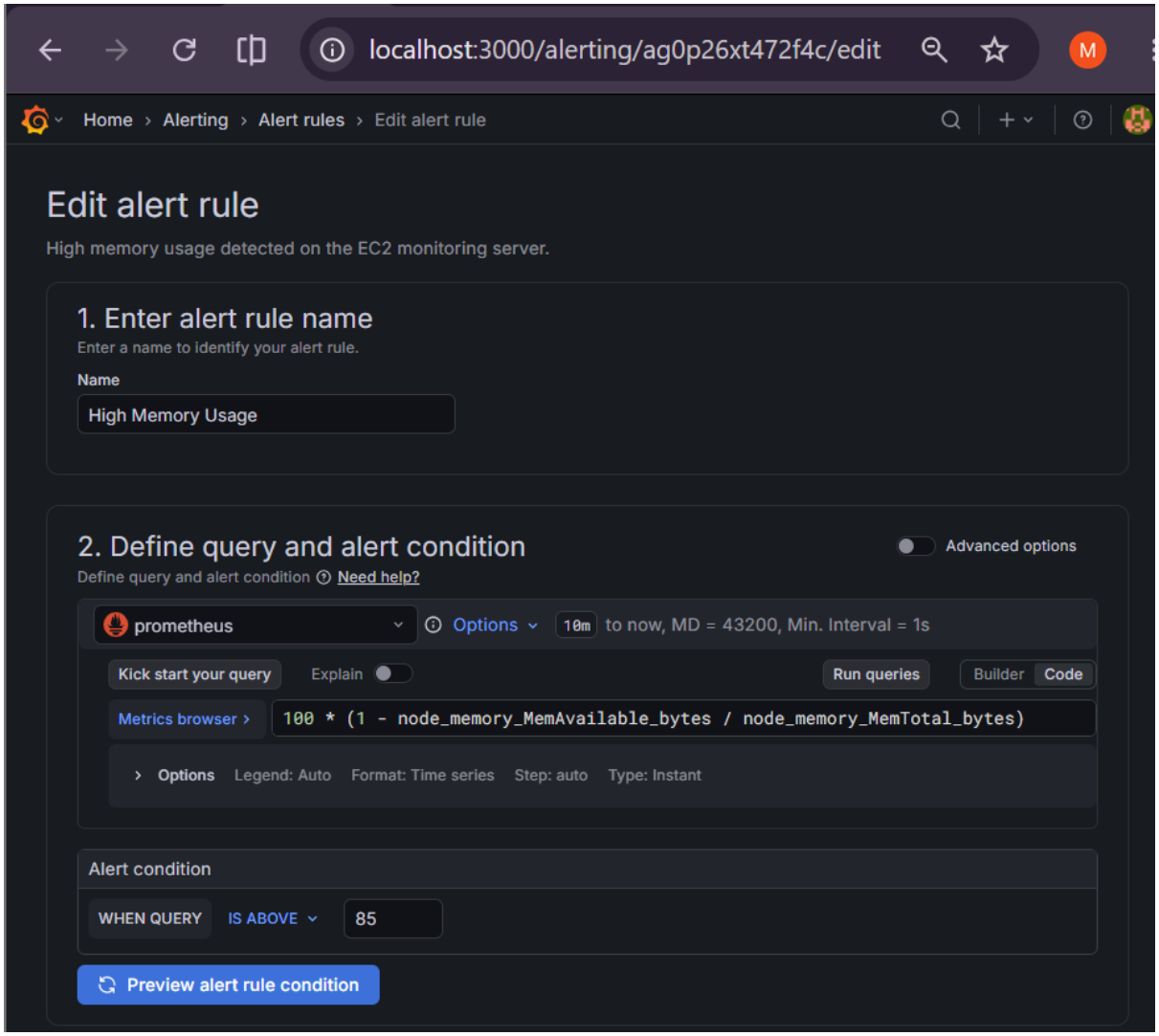
After testing, all three rules returned to their normal state.

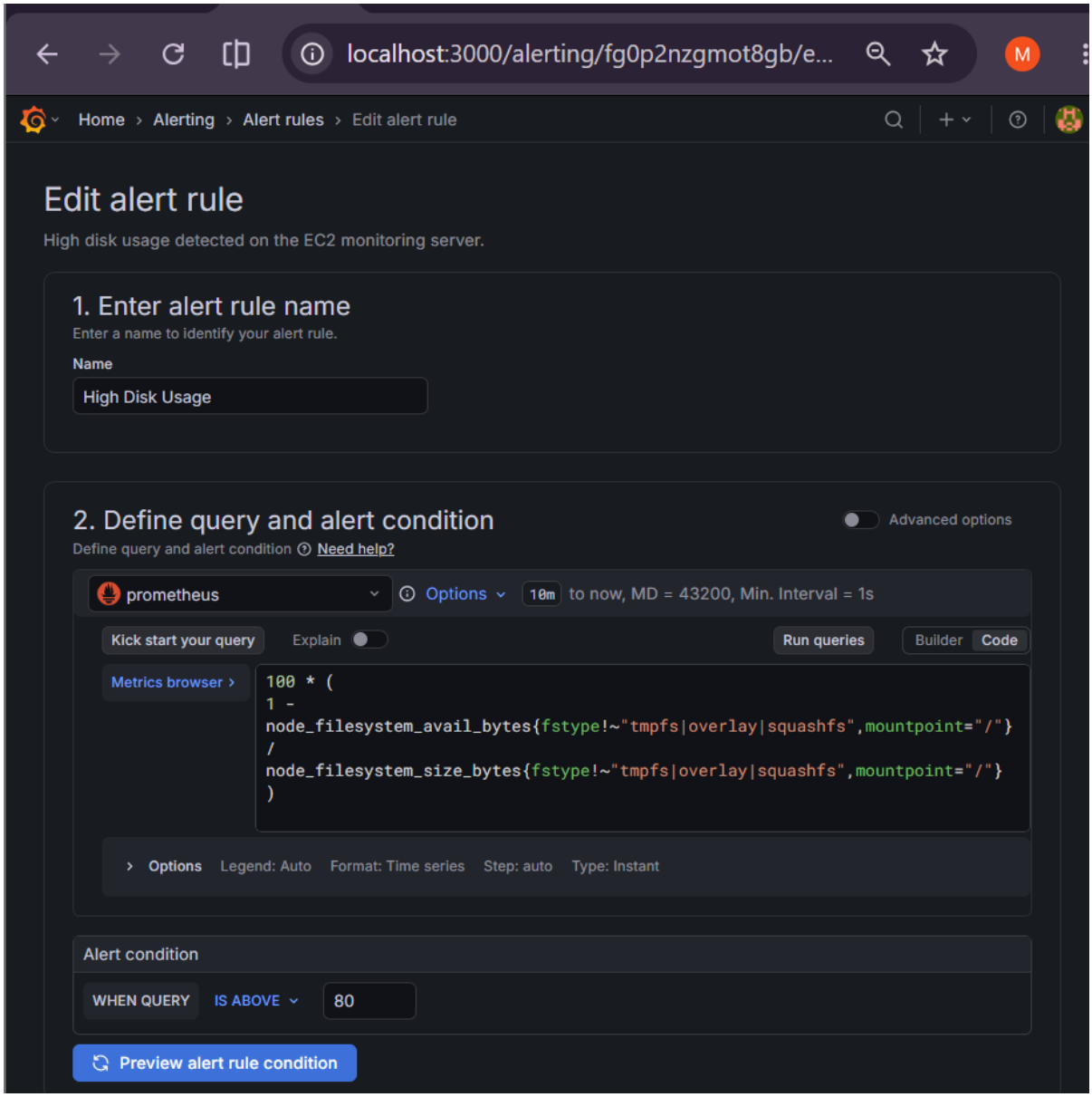
Screenshot 11: Grafana alert rules



Screenshot 12: Alert rule settings


localhost:3000/alerting/eg0p1dfedejnkb/edit
Home › Alerting › Alert rules › Edit alert rule
Edit alert rule
High CPU usage detected on the EC2 monitoring server.
1. Enter alert rule name
Enter a name to identify your alert rule.
Name
High CPU Usage
2. Define query and alert condition
Advanced options
Define query and alert condition Need help?
prometheus
Options
10m to now, MD = 43200, Min. Interval = 1s
Kick start your query
Explain
Run queries
Builder Code
Metrics browser ›
100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))
Options Legend: Auto Format: Time series Step: auto Type: Instant
Alert condition
WHEN QUERY
IS ABOVE
80
Preview alert rule condition


localhost:3000/alerting/ag0p26xt472f4c/edit
Home › Alerting › Alert rules › Edit alert rule
Edit alert rule
High memory usage detected on the EC2 monitoring server.
1. Enter alert rule name
Enter a name to identify your alert rule.
Name
High Memory Usage
2. Define query and alert condition
Advanced options
Define query and alert condition Need help?
prometheus
Options
10m
to now, MD = 43200, Min. Interval = 1s
Kick start your query
Explain
Run queries
Builder
Code
Metrics browser ›
100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)
Options
Legend: Auto
Format: Time series
Step: auto
Type: Instant
Alert condition
WHEN QUERY
IS ABOVE
85
Preview alert rule condition



13. Email Notification Configuration

Grafana was configured to send email notifications through Gmail SMTP.

SMTP configuration

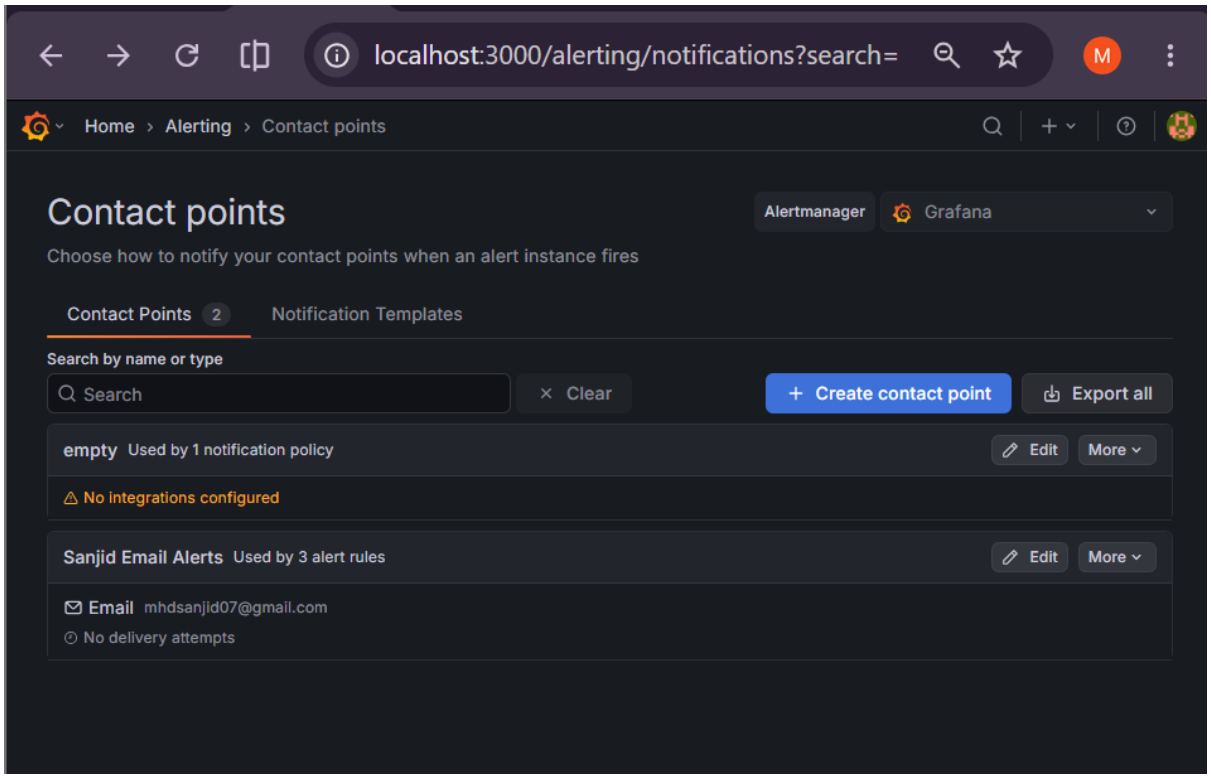
- SMTP host: `smtp.gmail.com:587`
- SMTP username: Configured Gmail account
- Sender name: `Grafana Alerts`
- Sender address: Configured Gmail address
- TLS certificate verification: Enabled

The Gmail App Password was stored in the `.env` file and referenced through an environment variable in Docker Compose. The actual password was not included in the documentation.

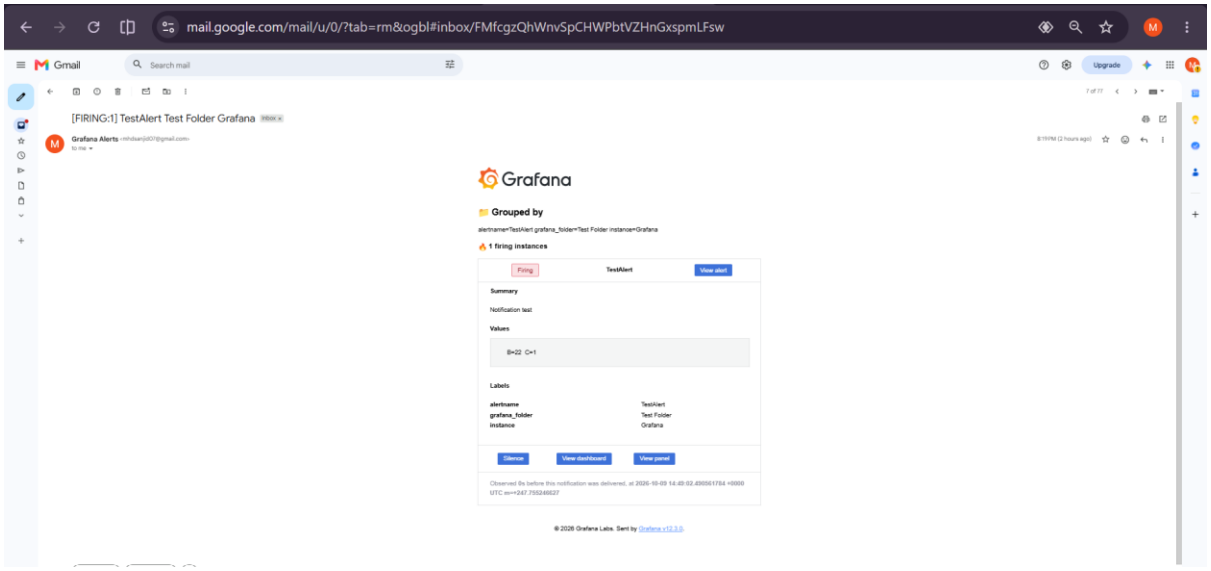
A Grafana contact point named `Sanjid Email Alerts` was created, and the intended recipient email address was configured.

The test notification was successfully delivered to the configured email inbox. All three alert rules were connected to the email notification destination.

Screenshot 13: Grafana contact point configuration



Screenshot 14: Test email received



14. Alert Testing and Verification

Each alert rule was tested separately by temporarily lowering its threshold so that the current resource utilization would trigger the condition.

After each alert fired and the corresponding email was received, the original threshold was restored.

Alert test	Temporary threshold	Result
High CPU Usage	0.5%	Alert fired; email received
High Memory Usage	50%	Alert fired; email received
High Disk Usage	50%	Alert fired; email received

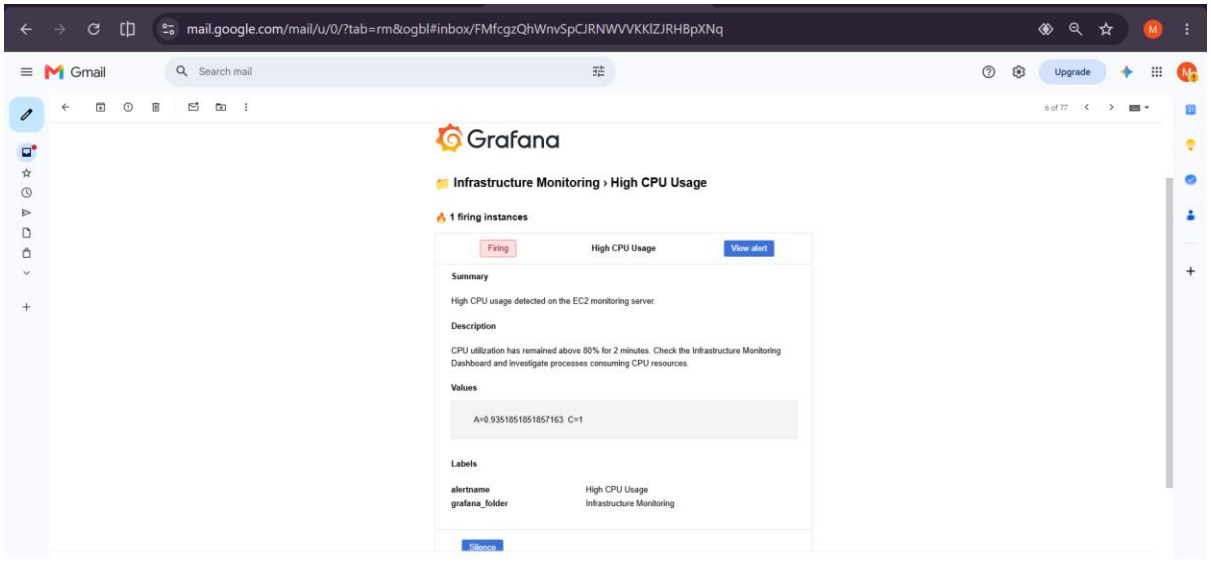
After testing, the original thresholds were restored:

- CPU utilization: 80%
- Memory utilization: 85%
- Disk utilization: 80%

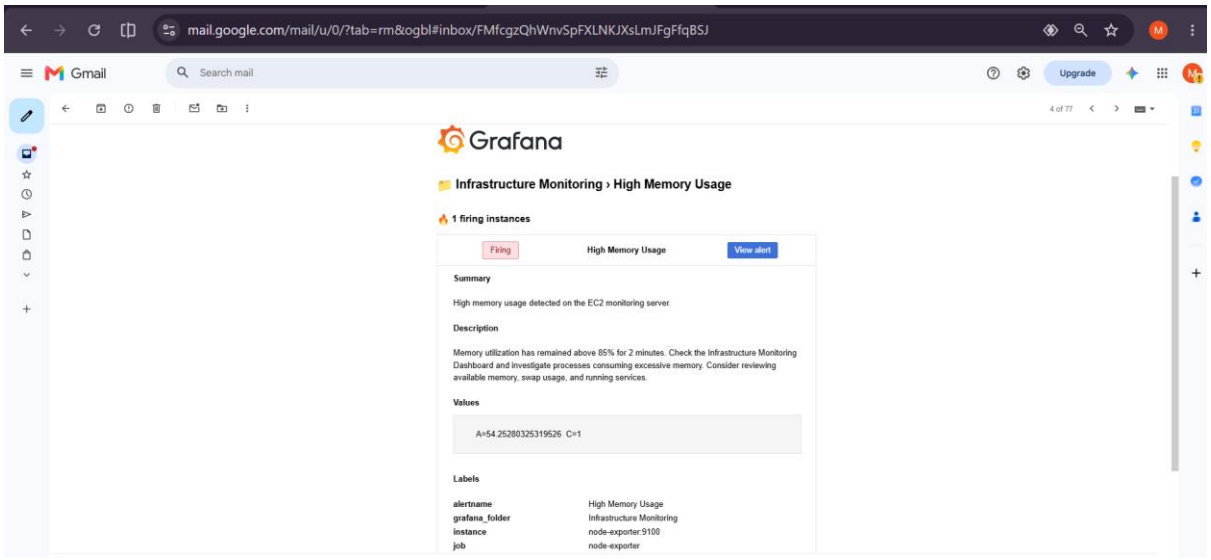
All three rules subsequently returned to their normal state.

These tests verified that the configured rules could trigger and deliver email notifications through the SMTP connection.

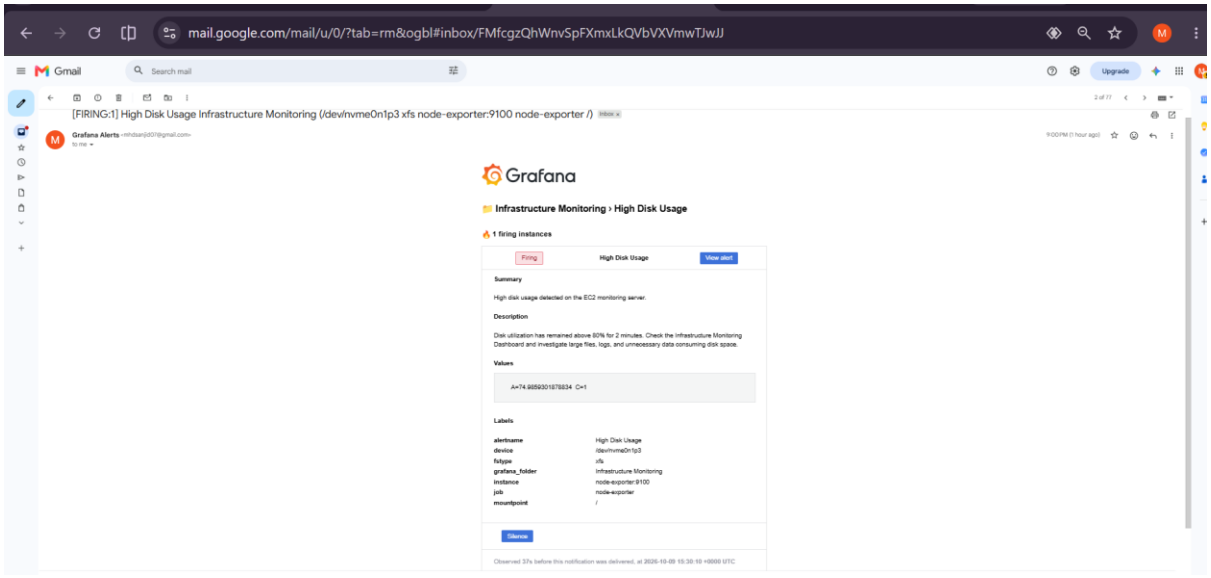
Screenshot 15: High CPU Usage email



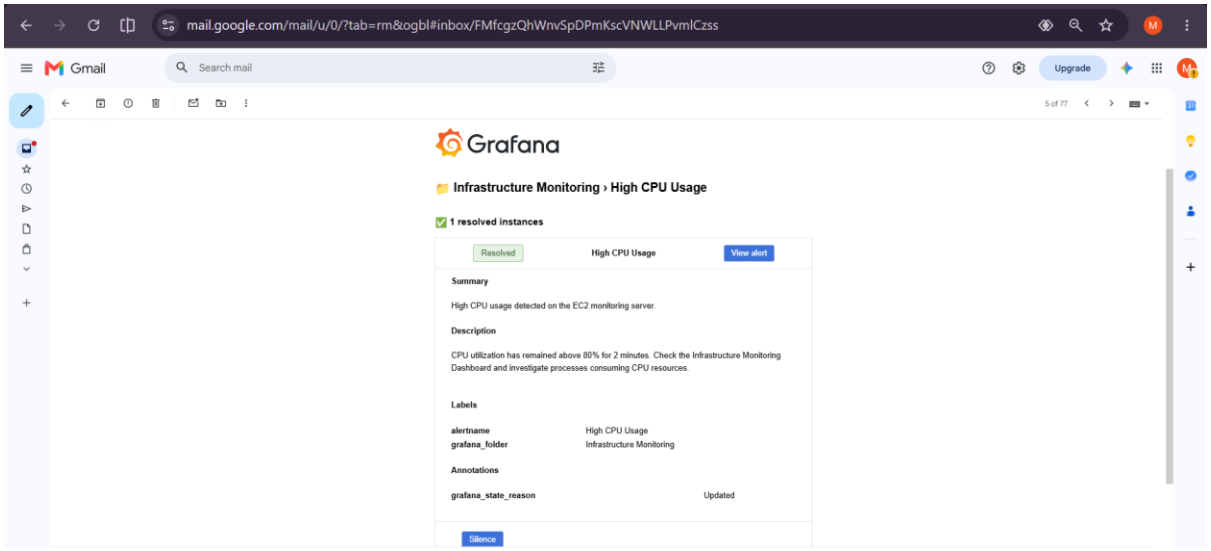
Screenshot 16: High Memory Usage email

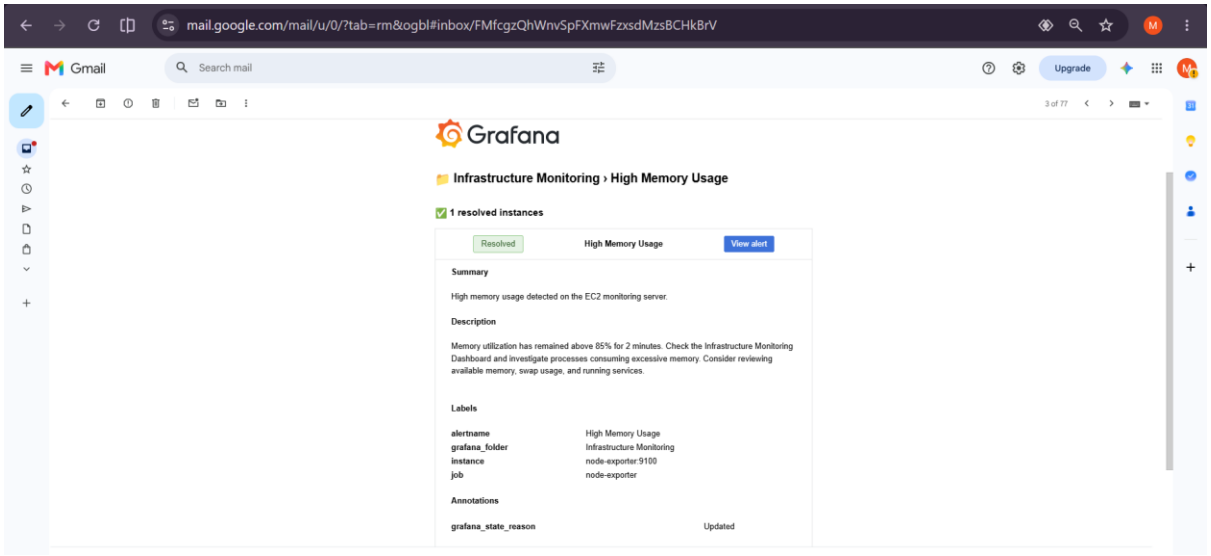


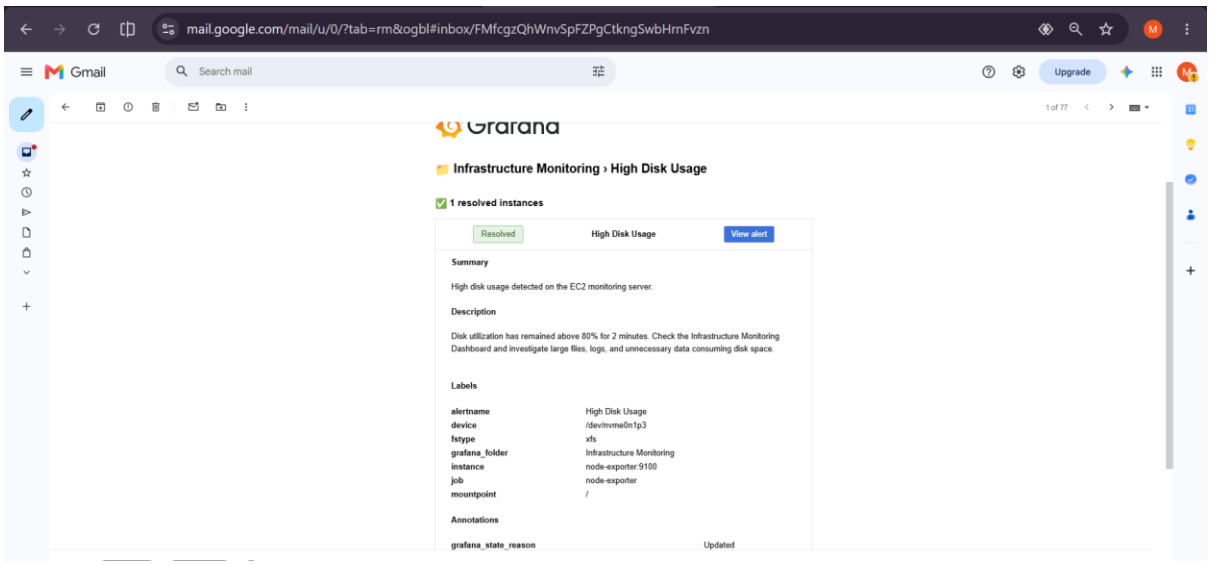
Screenshot 17: High Disk Usage email



Screenshot 18: Alert rules returned to Normal







15. Service Restart and Persistence Verification

Each monitoring service was restarted individually to verify service recovery.

15.1 Prometheus Restart

Prometheus was restarted using:

```
docker compose restart prometheus
```

The Prometheus health and target status were checked afterward. Both Prometheus and Node Exporter reported `health: up`, with no scrape errors.

15.2 Node Exporter Restart

Node Exporter was restarted using:

```
docker compose restart node-exporter
```

After allowing time for the next scrape, both monitoring targets were verified as healthy.

15.3 Grafana Restart

Grafana was restarted using:

```
docker compose restart grafana
```

The health endpoint returned a successful response containing:

```
{
  "database": "ok",
  "version": "12.3.0"
}
```

The Grafana dashboard was opened in the browser after the restart. All five panels remained available and working.

The dashboard's continued availability, along with the healthy database response, confirmed that the saved dashboard configuration remained accessible after the restart.

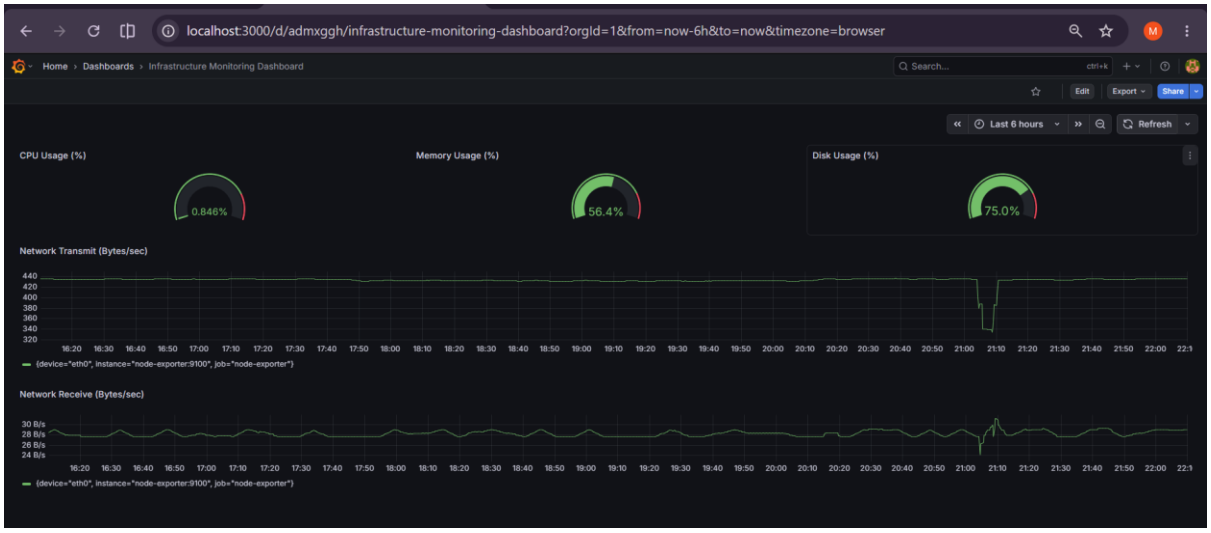
Screenshot 19: Prometheus targets after restart

```
[ec2-user@ip-172-31-14-161 monitoring-dashboard]$ curl -sS --max-time 10 http://127.
0.0.1:9090/api/v1/targets
{"status":"success","data":{"activeTargets":[{"discoveredLabels":{"__address__":"nod
e-exporter:9100","__always_scrape_classic_histograms__":"false","__convert_classic_h
istograms_to_nhcb__":"false","__metrics_path__":"/metrics","__scheme__":"http","__sc
rape_interval__":"30s","__scrape_native_histograms__":"false","__scrape_timeout__":"
10s","job":"node-exporter"},"labels":{"instance":"node-exporter:9100","job":"node-ex
porter"},"scrapePool":"node-exporter","scrapeUrl":"http://node-exporter:9100/metrics
","globalUrl":"http://node-exporter:9100/metrics","lastError":"","lastScrape":"2026-
10-09T16:38:36.672923897Z","lastScrapeDuration":0.012812631,"health":"up","scrapeInt
erval":"30s","scrapeTimeout":"10s"},{"discoveredLabels":{"__address__":"prometheus:9
090","__always_scrape_classic_histograms__":"false","__convert_classic_histograms_to
_nhcb__":"false","__metrics_path__":"/metrics","__scheme__":"http","__scrape_interva
l__":"30s","__scrape_native_histograms__":"false","__scrape_timeout__":"10s","job":"
prometheus"},"labels":{"instance":"prometheus:9090","job":"prometheus"},"scrapePool"
:"prometheus","scrapeUrl":"http://prometheus:9090/metrics","globalUrl":"http://prome
theus:9090/metrics","lastError":"","lastScrape":"2026-10-09T16:38:38.642586789Z","la
stScrapeDuration":0.005424748,"health":"up","scrapeInterval":"30s","scrapeTimeout":"
10s"}],"droppedTargets":[],"droppedTargetCounts":{"node-exporter":0,"prometheus":0}}
}[ec2-user@ip-172-31-14-161 monitoring-dashboard]$
```


Screenshot 20: Grafana health after restart


```
}[ec2-user@ip-172-31-14-161 monitoring-dashboard]$ curl -sS --max-time 10 http://127
.0.0.1:3000/api/health
{
  "database": "ok",
  "version": "12.3.0",
  "commit": "20051fb1fc604fc54aae76356da1c14612af41d0"
}[ec2-user@ip-172-31-14-161 monitoring-dashboard]$
```


Screenshot 21: Dashboard after restart



16. Persistent Storage and Backup

Docker named volumes were used to preserve application data across container restarts and recreation.

The configured volumes were:

- `prometheus-data`: Stores Prometheus time-series data.

- `grafana-data`: Stores Grafana data, including its database.

A Grafana database backup was maintained at:

`~/monitoring-dashboard/grafana.db.backup`

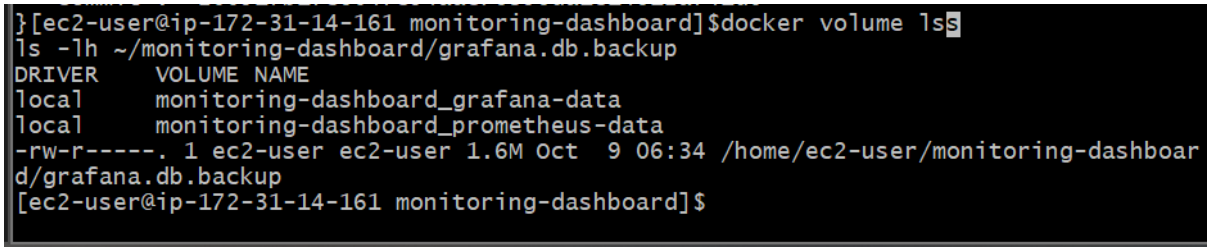
The restart tests were performed without removing the persistent volumes. The dashboard remained available afterward.

The following command was avoided during routine restart testing because the `-v` option removes Compose-managed volumes:

```
docker compose down -v
```

Persistent volumes and backups should be retained until a separate backup and recovery procedure has been tested.

Screenshot 22: Persistent storage or backup



17. Security Measures

The following security practices were applied:

1. Grafana and Prometheus were bound to `127.0.0.1` instead of being exposed directly to the public network.
2. The monitoring interfaces were accessed through an SSH tunnel.
3. Node Exporter's port was not published directly to the public network.
4. Grafana public user registration was disabled.
5. The `.env` file was protected with file permissions `600`.
6. Passwords, Gmail App Passwords, and private keys were excluded from the documentation.
7. Persistent volumes were retained during restart testing.
8. Resource limits and Prometheus retention settings were configured to help manage resource consumption.

The EC2 security group should allow only the required administration traffic. Grafana and Prometheus ports should remain closed to direct public access when the SSH tunnel method is used.

18. Troubleshooting

18.1 Grafana is inaccessible

Verify that the SSH tunnel is connected and the Grafana container is running.

```
docker compose ps
```

Reconnect the SSH tunnel if necessary.

18.2 Prometheus target is down

Check the container status and review the Prometheus logs.

```
docker compose ps
docker compose logs --tail=50 prometheus
```

Then recheck the targets in Prometheus.

18.3 Grafana data source connection fails

Verify that Prometheus is running and the data source URL is:

http://prometheus:9090

Use Grafana's **Save & test** option to recheck the connection.

18.4 Email notifications are not received

Check the Grafana logs:

```
docker compose logs --tail=100 grafana
```

Verify the SMTP host, username, App Password, sender address, and recipient. Do not share the App Password when sharing logs or screenshots.

18.5 Metrics are temporarily unavailable after a restart

Allow the services to recover and wait for at least one 30-second scrape interval. Then check the Prometheus targets and Grafana panels again.

19. Results

The monitoring and alerting environment was deployed successfully on AWS EC2 using Docker Compose.

The completed implementation included:

- Three running monitoring containers.
- Prometheus collection of host and self-monitoring metrics.
- A Grafana dashboard with five monitoring panels.
- Three resource utilization alert rules.
- Successful Gmail SMTP test notifications.
- Successful email delivery tests for all three alert rules.
- Healthy Prometheus targets after service restarts.
- A healthy Grafana database after restart.
- A dashboard that remained available after Grafana restarted.
- Persistent storage and basic security controls.

All original alert thresholds were restored after testing, and the three rules returned to their normal state.



20. Conclusion

The Infrastructure Monitoring and Alerting Dashboard demonstrated how open-source monitoring tools can be deployed together on an AWS EC2 instance to monitor system resources and notify administrators about potential resource issues.

Prometheus collected infrastructure metrics through Node Exporter, while Grafana provided a centralized dashboard and alerting interface. Gmail SMTP delivered notifications when the alert conditions were triggered during testing.

The restart tests confirmed that the services recovered successfully and the saved dashboard remained available. The project provided practical experience with Docker Compose, PromQL, infrastructure monitoring, alert configuration, email notifications, persistent storage, and secure remote access.

The implementation provides a foundation for future improvements, including monitoring additional servers, expanding dashboard metrics, and implementing automated backup and recovery procedures.